

Reasoning Matters for 3D Visual Grounding

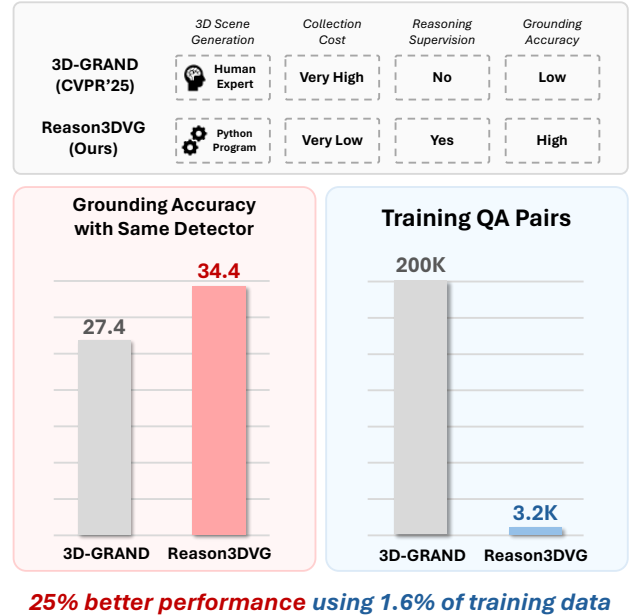
Hsiang-Wei Huang Kuang-Ming Chen Wenhao Chai Cheng-Yen Yang
Jen-Hao Cheng Jenq-Neng Hwang
University of Washington
{hwhuang, kmchen, wchai, cycyang, andyhci, hwang}@uw.edu

Abstract

The recent development of Large Language Models (LLMs) with strong reasoning ability has driven research in various domains such as mathematics, coding, and scientific discovery. Meanwhile, 3D visual grounding, as a fundamental task in 3D understanding, still remains challenging due to the limited reasoning ability of recent 3D visual grounding models. Most of the current methods incorporate a text encoder and visual feature encoder to generate cross-modal fuse features and predict the referring object. These models often require supervised training on extensive 3D annotation data. On the other hand, recent research also focus on scaling synthetic data to train stronger 3D visual grounding LLM, however, the performance gain remains limited and non-proportional to the data collection cost. In this work, we propose a 3D visual grounding data pipeline, which is capable of automatically synthesizing 3D visual grounding data along with corresponding reasoning process. Additionally, we leverage the generated data for LLM fine-tuning and introduce Reason3DVG-8B, a strong 3D visual grounding LLM that outperforms previous LLM-based method 3D-GRAND using only 1.6% of their training data, demonstrating the effectiveness of our data and the importance of reasoning in 3D visual grounding.

1. Introduction

3D Visual Grounding is a fundamental task in 3D understanding, aiming to identify a target object within a 3D scene based on a given textual query. Recent supervised-training models [16, 21, 25, 26, 28, 35] integrate a text encoder and a visual encoder to generate cross-modal features for target object prediction. These supervised methods rely on large-scale, real-world annotated 3D visual grounding datasets for supervised training. To mitigate this, some recent research also explores approaches that leverage proprietary Large Language Models (LLMs) and Vision Language Models (VLMs) for the 3D visual grounding task via an agentic workflow [18, 29] or through in-context ex-



25% better performance using 1.6% of training data

Figure 1. Our method outperforms 3D-GRAND on the ScanRefer benchmark when using only **1.6%** amount of their training data scale. Compared with 3D-GRAND, our proposed data pipeline features lower data collection cost, incorporate extra reasoning supervision for LLM, and achieve **25%** better grounding accuracy.

amples and code generation [34]. Despite these methods somewhat demonstrating success in the 3D visual grounding task, they heavily rely on proprietary LLMs and VLMs to achieve the best visual grounding performance, which introduce extensive inference cost during test time.

To advance open-source models for improved performance on 3D visual grounding, recent work also investigates the direction of collecting 3D visual grounding data for open-source LLM fine-tuning. 3D-GRAND [30] collects a million-scale 3D visual grounding dataset and fine-tunes open-source LLM on this large-scale data. Despite showing performance improvements on public benchmarks, several new challenges emerge. First, 3D-GRAND relies

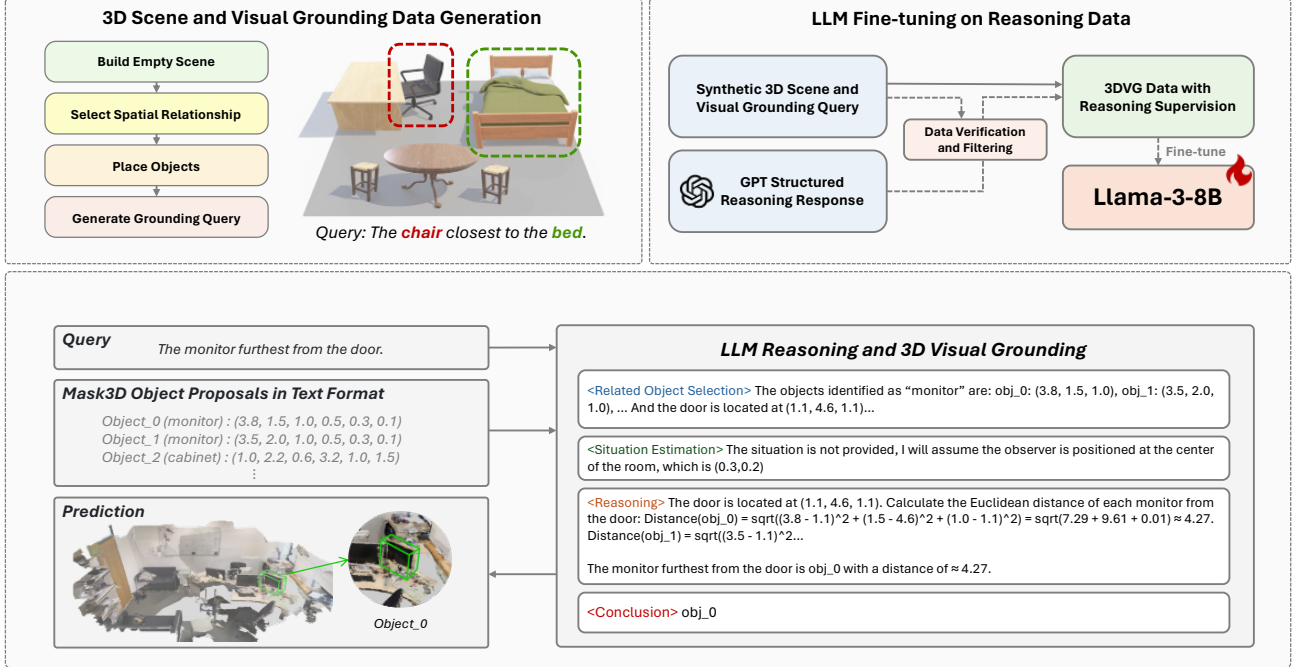


Figure 2. We propose a fully automatic data pipeline that can generate visual grounding queries and reasoning responses. The collected data are used to conduct LLM fine-tuning, which results in Reason3DVG-8B, a powerful LLM with strong 3D visual grounding ability.

on human-expert-designed 3D scenes, which require extensive manual labor. In addition, the dense object-level annotations introduce further cost. Furthermore, their experimental results also indicate that the performance improvement achieved through fine-tuning on such large-scale data is moderate and not proportional to the significant data collection effort. These challenges motivate us to rethink 1) a more cost-efficient, fully automatic data pipeline that can generate 3D visual grounding data in a human-free and cost-efficient fashion, and 2) the actual key beyond data scale towards improving the performance of open-source LLM on the challenging 3D visual grounding task.

In this work, we explore the direction of improving the LLM’s 3D visual grounding performance using fully automatically generated synthetic data, featuring detailed, structured reasoning supervision for LLM fine-tuning. Our proposed automated data collection pipeline introduces minimal cost, featuring detailed, structured reasoning supervision. LLM fine-tuned on our data outperforms previous LLM-based method 3D-GRAND that trained on $60\times$ more data, as shown in Fig. 1.

Our work first introduces a fully automatic, 3D visual grounding data pipeline, which aims to address the dependency on extensive human-annotated 3D visual grounding data. We use our collected data and conduct LLM fine-tuning, resulting in Reason3DVG, an LLM with strong reasoning ability and advanced 3D visual grounding accuracy. We perform extensive evaluation and show that Rea-

son3DVG achieves superior grounding accuracy than SOTA zero-shot methods and LLM-based method 3D-GRAND on multiple 3D visual grounding benchmarks including ScanRefer and NR3D. We summarize the main contribution of our work as follows:

- We propose a fully automatic 3D visual grounding data pipeline for LLM fine-tuning. Our data pipeline does not require any human annotation, which largely reduces the data collection cost compared to previous works.
- We conduct LLM fine-tuning on our collected data and introduce Reason3DVG, an LLM for 3D visual grounding task that achieves strong accuracy on multiple 3D visual grounding benchmarks including ScanRefer and NR3D.
- Our model outperforms previous 3D visual grounding LLM 3D-GRAND when using only 1.6% of training data, demonstrating the importance of reasoning supervision in the 3D visual grounding task, and serves as a cornerstone for future 3D understanding LLM development with stronger reasoning and understanding ability.

2. Related Work

3D visual grounding is a fundamental 3D understanding task that aims to predict the target object in the 3D scene based on a given language query. Most of the recent 3D visual grounding methods [14, 16, 19, 28] adopt separate encoders for each input modality such as text and point cloud to generate cross-modal features and conduct pre-

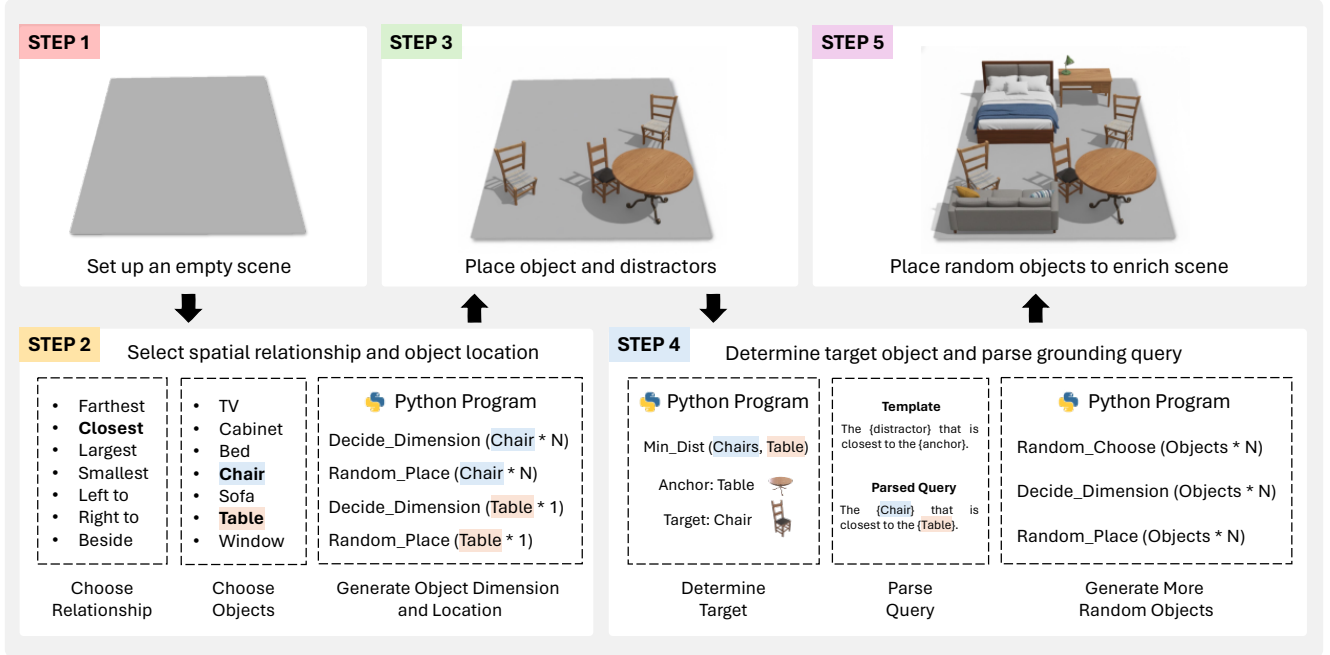


Figure 3. An illustration of our 3D scene layout generation pipeline, which includes 5 steps: **1)** Set up an empty 3D scene with certain dimension. **2)** Choose a spatial relationship and decide the anchor and distractor objects as well as their dimension and location. **3)** Place them in the 3D scene. **4)** Determine the target object from the distractors and query. **5)** Generate more distractor objects to enrich 3D scene.

diction. Some end-to-end methods like SAT [32], LanguageRefer [22], and UniT3D [5] utilized a unified multi-modal transformer to conduct 3D visual grounding. Most of these methods utilize supervised training on human collected 3D data, which incur extensive annotation efforts. Recent works started to focus on the LLM-based approaches [29, 34], which leverage proprietary LLM to solve the 3D visual grounding task, but introduce extensive inference costs. 3D-GRAND [30] attempts to scale-up the training data for small-scale, open-source LLM training, yet it still requires extensive 3D scene layout annotation, and only achieve sub-optimal grounding performance. In this work, we explore using an open-source LLM to solve the 3D visual grounding task through our fully automatic data collection pipeline, which does not require any human expert annotations, and outperforms existing LLM-based methods on multiple 3D visual grounding benchmarks.

3. Method

We illustrate our proposed framework in Fig. 2. We design an automatic 3D visual grounding data pipeline that features detailed, structured reasoning responses for LLM fine-tuning. We fine-tuned open-source LLM Llama-3.1-8B on our collected data. During test time, we utilize an object detector Mask3D to generate object proposals, which are transformed to text format and sent to the LLM along with the query. The LLM then predicts the referring target object

by performing multi-stage structured reasoning.

3.1. 3D Scene Generation

The collection of 3D scene data requires a huge effort, including extensive human expert annotations and indoor 3D scan collection. To address this, we propose to generate synthetic 3D scene data for LLM-based model training and enable the model to conduct 3D visual grounding in real-world 3D scenes. There are many existing methods for 3D scene generation [9, 31], but they mostly focus on generating scene layout with realistic object arrangements or following the specified room style from users’ prompt, while our goal is to collect object-centric 3D scene that focuses on objects’ spatial arrangements as well as the target object’s corresponding grounding query, which is an important component of 3D visual grounding data.

In this work, we design a program-based 3D scene data pipeline that can generate an object-centric 3D scene following several common spatial relationships between objects (shown in Fig. 3-Step.2). Here, we provide a generation example of a 3D scene layout in Fig. 3, which contains the 3D scene layout following query “The Chair that is closest to the Table.” with n distractor chairs in the scene:

1. Define a 3D scene size with a certain dimension $H \times W$.
2. Place n chairs and one table in the 3D scene.
3. Calculate the distance between each chair and table to find the closest chair to the table; this chair will be the

target object of the 3D visual grounding query.

4. Generate more objects to enrich the 3D scene diversity. Following this procedure, we collect a wide range of 3D scene layouts that follow our specified common spatial relationship and are not confined by the limited number of 3D scene layouts in the real-world available 3D scans. For each spatial relationship, we design multiple templates from which we can parse the target’s object class to generate the final target object query. We prompt GPT-4o to generate some indoor scene objects as well as their common dimension. When placing objects, we randomly select object classes and apply a random variation on their size to enhance diversity. We attached our code for 3D scene generation, as well as all our generated 3D scenes in the supplemental materials for reference.

It is important to note that our data pipeline **are not designed to generate complex queries and spatial relationships that simulate the real-world complexity**. This is because recent LLMs already possess basic reasoning ability; our data utilizes several common spatial relationships with detailed reasoning responses serving as supervision to encourage the LLM to think step-by-step in a chain-of-thought manner for the 3D visual grounding task. Furthermore, our experiments and ablation study results show that after fine-tuning on our simple spatial relationships, **the LLM can solve complex, real-world 3D visual grounding queries that were not seen in our collected data**, as suggested in our study in Tab. 5. Lastly, because our goal is to generate 3D scene data that follows our specified spatial relationship between objects, we only refrain the object from being placed overlapping during scene generation, and do not focus on placing them in a realistic manner.

3.2. Four Stages Reasoning

Recent advanced LLMs with reasoning ability adopt special tokens or tags [10, 24] to separate the reasoning and conclusion stage, which enables a more structured and organized thinking process of the LLM. In the 3D visual grounding domain, it is also important to think step-by-step in a chain-of-thought manner before generating the final prediction, especially when trying to interpret the complicated spatial relationship in the target query. To achieve this, we proposed four structured reasoning stages, as illustrated in our Fig. 2. These four chain-of-thought reasoning stages encourage the LLM to generate a more structured thinking process. The four stages include:

- **Related Object Selection.** Given all the objects in the 3D scene, the LLM listed the objects that might be relevant to the given query. Each listed object will include information such as its object ID, class name, location, and dimension.
- **Situation Estimation.** If a 3D situation (viewer’s location) is provided, the LLM will estimate its possible co-

ordinate in the 3D scene. Otherwise, the LLM defaults to assume it is situated in the middle of the scene.

- **Reasoning.** This is the main stage for LLM to conduct reasoning based on the previously given information. The LLM will conduct necessary math calculations and logical reasoning to derive the final answer.
- **Conclusion.** In this stage, the model outputs the final prediction in a structured format, which enables easier parsing, evaluation and comprehension by the user.

3.3. Reasoning Data Collection

To collect detailed reasoning data aligned with our four reasoning stages, we leverage proprietary LLM GPT-4o to generate structured reasoning responses. We design a specialized prompt to guide GPT-4o in adhering to our four-stage reasoning process. The full prompt and the 3D scene input format is provided in the appendix. We send our collected queries and corresponding 3D scenes in structured text format to GPT-4o to generate reasoning response. Each sample includes a unique 3D scene layout with over 50 objects, a target object’s language query, its corresponding object ID, and the reasoning response from GPT-4o.

After data collection, we further perform a verification process to ensure the quality of our dataset. Since each 3D scene layout is generated through rule-based code, the layout is **guaranteed** to correctly correspond to the specified spatial relationships. For the GPT-4o generated responses, we further validate the final predicted answer against our generated ground truth and remove any responses with incorrect predictions. Approximately 10% of responses that deviate from our specified format or contain incorrect answers are automatically filtered through string pattern matching and answer verification. The remaining verified reasoning responses are incorporated into our final training set, which contains 3.2K samples.

To further measure the correctness of the reasoning process, we also conduct manual verification on a subset of the responses. We observe that most major reasoning steps are correct, with only minor arithmetic errors (typically after the first decimal place) that do not affect the correctness of final predictions. Therefore, we do not apply additional filtering based on intermediate reasoning results and instead use the final answer as our main filtering criterion.

3.4. 3D Visual Grounding LLM Fine-tuning

We employ the open-source LLM Llama-3.1-8B [7] as our base model. The base model Llama-3.1-8B is fine-tuned on our collected 3.2K 3D visual grounding reasoning data. The model is supervised by the step-by-step reasoning responses generated by GPT-4o, using the standard cross-entropy loss with the LLM’s Next-Token-Prediction (NTP) objective, which can be formally defined as:

Table 1. Performance comparison with existing methods on ScanRefer validation set. We follow previous work and report the grounding accuracy under different IoU threshold, and also report the accuracy of “Unique” (scenes with a single target object) and “Multiple” (scenes with distractors of the same class) subsets, along with overall performance.

Method	Source	Unique		Multiple		Overall	
		Acc@0.25	Acc@0.5	Acc@0.25	Acc@0.5	Acc@0.25	Acc@0.5
<i>Fine-tuned on Training Set</i>							
ScanRefer [3]	ECCV’20	67.6	46.2	32.1	21.3	39.0	26.1
InstanceRefer [33]	ICCV’21	77.5	66.8	31.3	24.8	40.2	32.9
3DVG-Transformer [35]	ICCV’21	77.2	58.5	38.4	28.7	45.9	34.5
BUTD-DETR [16]	ECCV’22	84.2	66.3	46.6	35.1	52.2	39.8
EDA [28]	CVPR’23	85.8	68.6	49.1	37.6	54.6	42.3
3D-VisTA [36]	ICCV’23	81.6	75.1	43.7	39.1	50.6	45.8
G3-LQ [26]	CVPR’24	88.6	73.3	50.2	39.7	56.0	44.7
MCLN [21]	ECCV’24	86.9	72.7	52.0	40.8	57.2	45.7
ConcreteNet [25]	ECCV’24	86.4	82.1	42.4	38.4	50.6	46.5
<i>Not Fine-tuned on Training Set</i>							
LRF [17]	ICCV’23	-	-	-	-	4.8	0.9
OpenScene [20]	CVPR’23	20.1	13.1	11.1	4.4	13.2	6.5
LLM-Grounder [29]	ICRA’24	-	-	-	-	17.1	5.3
WS-3DVG [27]	ICCV’23	-	-	-	-	27.4	22.0
ZSVG3D [34]	CVPR’24	63.8	58.4	27.7	24.6	36.4	32.7
3D-GRAND [30]	CVPR’25	54.4	36.4	26.0	20.8	38.0	27.4
Reason3DVG-8B	Ours	76.6	69.5	31.1	27.3	38.7	34.4

$$\mathcal{L}_{\text{CE}} = - \sum_{t=1}^T \log P_{\theta}(y_t | y_{<t}, x) \quad (1)$$

where x denotes the input (3D object proposals in text format and target query), $y_{1:T}$ is the ground-truth response sequence from GPT-4o, and P_{θ} is the model’s predicted probability distribution over tokens at each time step t . Each training sample includes the 3D scene context, a target object query, and a structured four-stage reasoning response. By learning from these detailed reasoning sequences, the model is encouraged not only to predict the correct answer but also to explicitly generate interpretable intermediate reasoning steps.

3.5. Inference Pipeline

During test time, we follow previous non-fine-tuned methods [18, 30, 34] and use 3D object detector Mask3D [23] that takes point clouds as input to generate object proposals and corresponding class labels. These proposals are parsed into a structured text format, with each object assigned a unique object identifier ID. The object proposals are fed into our Reason3DVG along with the given language query. Reason3DVG then conduct structured reasoning and predict the referring target object’s ID. An example of the input format and inference prompt can be found in our appendix.

4. Experiments

4.1. Implementation Details

We use Llama-3.1-8B [7] as our base model and conduct fine-tuning using our collected 3D visual grounding reasoning data, which consists of 3.2K 3D scenes, target queries, and structured reasoning responses. More details on training hyperparameters and implementation setup can be found in our appendix.

4.2. Benchmarks

We evaluated on two large-scale 3D visual grounding benchmarks, including ScanRefer [3] and NR3D [2] following previous works [18, 29, 30, 34]. Both benchmarks leverage 3D scenes collected from ScanNet [6]. We follow previous zero-shot methods [18, 30, 34] and use Mask3D [23] to obtain the same 3D bounding boxes and the object class label for consistent evaluation and fair comparison.

ScanRefer. ScanRefer provides 51,500 natural language descriptions from 800 different 3D scenes. We follow previous works [16, 29, 34] and adopt the validation set of ScanRefer for evaluation, which contains 9,508 language queries. We report ScanRefer’s standard evaluation metrics: Accuracy@0.25 and Accuracy@0.5, where 0.25 and 0.5 are different IoU thresholds of 3D bounding boxes.

Table 2. Performance on NR3D. Queries are labeled as “Easy” (one distractor) or “Hard” (multiple distractors), and as “View-Dependent” or “View-Independent” based on viewpoint requirements for grounding. [†]We list the performance using oracle class label to provide reference for the upper bound performance.

Method	Source	Easy	Hard	Dep.	Indep.	Overall
<i>Fine-tuned on Training Set</i>						
ReferIt3DNet [2]	ECCV’20	43.6	27.9	32.5	37.1	35.6
TGNN [13]	AAAI’21	44.2	30.6	35.8	38.0	37.3
3DRefTransformer [1]	WACV’22	46.4	32.0	34.7	41.2	39.0
InstanceRefer [33]	ICCV’21	46.0	31.8	34.5	41.9	38.8
FFL-3DOG [8]	ICCV’21	48.2	35.0	37.1	44.7	41.7
LanguageRefer [22]	CoRL’22	51.0	36.6	41.7	45.0	43.9
3DVG-Transformer [35]	ICCV’21	48.5	34.8	34.8	43.7	40.8
TransRefer3D [11]	MM’21	48.5	36.0	36.5	44.9	42.1
BUTD-DETR [16]	ECCV’22	60.7	48.4	46.0	58.0	54.6
<i>Not Fine-tuned on Training Set</i>						
WS-3DVG [27]	ICCV’23	27.3	18.0	21.6	22.9	22.5
ZSVG3D [34]	CVPR’24	46.5	31.7	36.8	40.0	39.0
SeeGround (InternVL2-8B) [18]	CVPR’25	43.6	25.8	32.6	35.4	34.3
SeeGround (InternVL2-26B) [18]	CVPR’25	46.8	29.8	34.7	39.8	38.0
Reason3DVG-8B	Ours	50.4	31.0	37.3	42.0	40.4
Reason3DVG-8B[†]	Ours	63.0	36.7	40.1	54.5	49.3

NR3D. NR3D contains 41.5K natural language captions collected by humans, featuring multiple spatial relationships. Unlike ScanRefer, NR3D provides the GT 3D bounding boxes in the scene along with their corresponding object IDs. Given a language query, the visual grounding model is required to predict the target object ID that the given query refers to. We adhere to the original NR3D benchmark’s setup, categorizing text queries into “Easy” (scenarios with only one same-class distractor) and “Hard” (scenarios with multiple same-class distractors). Additionally, queries are also classified as “View-Dependent” or “View-Independent,” depending on whether specific viewpoints are necessary for correctly identifying the target object.

4.3. Comparison with Non-fine-tuned Methods

We compare our performance with other SOTA methods on two large-scale 3D visual grounding benchmarks, including ScanRefer (Tab.1) and NR3D (Tab. 2). On ScanRefer, our model outperforms the recent zero-shot methods ZSVG3D [34] and 3D-GRAND [30]. Notably, unlike ZSVG3D, which requires multiple in-context examples and proprietary GPT-4 during inference, our model conducts reasoning and prediction without any in-context ex-

amples. Compared with 3D-GRAND, we achieve better performance when training on much less data (3D-GRAND is trained on **200K** QA pairs, while our model is only trained on 3.2K QA pairs). On NR3D, our model outperforms recent zero-shot SOTA methods ZSVG3D and SeeGround [18] when using the same bounding box proposals from Mask3D as input. Furthermore, these two methods utilize extra visual input to achieve more fine-grained 3D scene understanding, while our model achieves better results by only using the object bounding box proposals in text format. Incorporating extra object visual features might potentially further improve our method’s performance.

4.4. Comparison with Fine-tuned Methods

Despite SOTA accuracy among non-fine-tuned methods and outperforming multiple fine-tuned baselines [1, 2, 13, 33], our results still lag behind SOTA models that are directly fine-tuned on real-world, large-scale data. We investigate the reason for such a gap by using the oracle object class label as input for NR3D, and found that our accuracy can be largely improved and even achieve comparable performance with SOTA fine-tuned models. This suggests that fine-tuned methods can obtain better detection and visual

Table 3. Ablation study on the effectiveness of fine-tuning. We report accuracy (%) on Easy, Hard, Depth-dependent (Dep.), Depth-independent (Indep.), and Overall splits.

Setup	Easy	Hard	Dep.	Indep.	Overall
Llama-3.1-8B	44.6	22.7	31.1	34.5	33.3
+ Fine-tuning	63.0 (+18.4)	36.7 (+14.0)	40.1 (+9.0)	54.5 (+20.0)	49.3 (+16.0)

Table 4. Ablation study on the impact of reasoning supervision during fine-tuning on NR3D. We report accuracy (%) across Easy, Hard, Depth-dependent (Dep.), Depth-independent (Indep.), and Overall splits.

Reasoning supervision	Easy	Hard	Dep.	Indep.	Overall
	42.2	25.4	30.2	35.4	33.5
✓	63.0 (+20.8)	36.7 (+11.3)	40.1 (+9.9)	54.5 (+19.1)	49.3 (+15.8)

Table 5. Ablation on domain generalization after fine-tuning. We report accuracy (%) on In-Domain and Out-of-Domain queries.

Model	In-Domain	Out-of-Domain
Llama-3.1-8B	34.5	32.1
+ Fine-tuning	49.6 (+15.1)	49.1 (+17.0)

features from the direct supervision of large-scale, real-world data, while our accuracy is limited by the detection quality. Providing the LLM richer semantic clues for reasoning with more sophisticated detectors or captioner [4] serves as a promising way to improve the performance.

4.5. Ablation Studies

Effectiveness of fine-tuning. To show the effectiveness and contribution of our data, we compare the performance on NR3D after fine-tuning. In Tab. 3, the model’s performance increased by a notable margin across all query categories after fine-tuning, with the overall accuracy increased by 16%, showing the effectiveness of our data.

Importance of reasoning supervision. We trained the base model on our collected data without the reasoning process to verify the importance of reasoning supervision. In this training setting, the model directly generates the final prediction without the reasoning process. This training strategy follows existing 3D LLM methods like 3D-GRAND [30] and 3D-LLM [12], which directly fine-tune the LLM with short and concise answers without reasoning process. Table. 4 showcases that involving the reasoning process during training can largely improve the performance, which demonstrates the importance of reasoning.

Generalization to out-of-domain queries. Our collected data includes only seven common in-domain spatial relationships; the goal is not to mimic the real-world complex-

ity, but some simple data to supervise the LLM to think step-by-step in the 3DVG task. To verify the LLM’s generalization ability on unseen and complex real-world queries, we examine the grounding accuracy improvement of our model on in-domain and out-of-domain queries after fine-tuning on our collected data. We found that despite the distribution shift to the complex real-world and unseen, novel spatial relationship, our model still demonstrates strong generalization ability, with more than 15% performance improvements on both in-domain and out-of-domain unseen queries, as shown in Tab. 5.

Fine-tuning with varied data scale. In our experiments, we found that training the LLM on a subset of our data can achieve comparable performance when fine-tuned for more iterations. This result confirms our hypothesis that recent LLMs like Llama-3 already possess basic reasoning abilities. Fine-tuning on our collected small-scale data benefits the LLM’s capability to reason step-by-step following our four-stage format, which improves the LLM’s accuracy on 3D visual grounding task and further enables it to generalize to unseen spatial relationships.

4.6. Qualitative Results

We provide grounding qualitative results in Fig. 4. We include multiple results from in-domain query, which has similar spatial relationships in our training data, and two out-of-domain queries that have unseen and more complex spatial relationships. The qualitative examples demonstrate the effectiveness of our fine-tuning strategy to enable LLM to conduct 3D visual grounding on the complex language queries and 3D scenes in real-world scenarios.

5. Limitations and Future Work

Despite our SOTA results, our accuracy is primarily limited by the quality of object proposals generated by the 3D object detector. Incorporating better detectors and object



When facing the desks choose the one on the right.
(In-domain query)

To determine the object ID based on the given description, we need to identify the desks and then choose the one on the right.

The desks are obj_10 and obj_11. Since obj_11 is on the right of obj_10, the object ID is obj_11.

Answer: **obj_11.**

Base Model

The objects that are desks are "obj_10" and "obj_11". Their coordinates are:

- obj_10: 2.8, 4.1, 0.8
- obj_11: 1.3, 4.0, 1.1

Assume my current location is at the center of the floor. The coordinates are:

- obj_14: 2.4, 3.3, 0.5

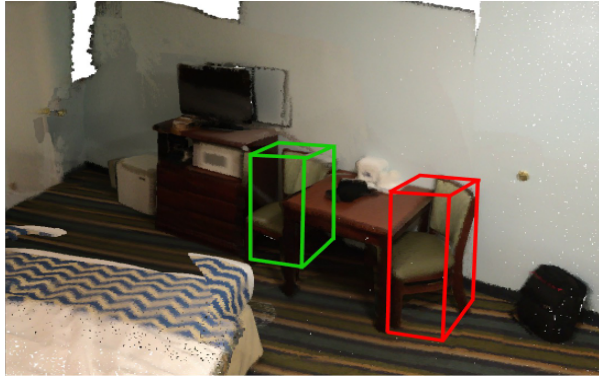
To determine which desk is on the right from my current location:

1. Calculate vectors from my position (O) to the desks:
 - Vector OA for obj_10: $(2.8 - 2.4, 4.1 - 3.3) = (0.4, 0.8)$
 - Vector OB for obj_11: $(1.3 - 2.4, 4.0 - 3.3) = (-1.1, 0.7)$
2. Calculate the cross product $OA \times OB$ to determine relative position:
 - Cross product $C = (0.4 * 0.7) - (0.8 * -1.1) = 0.28 + 0.88 = 1.16$

Since the cross product C is positive, obj_10 is on the right, and obj_11 is on the left.

Answer: **obj_10.**

Reason3DVG



The chair that is next to the TV, it is between the table and the tv.
(Out-of-domain, unseen query)

To find the object ID of the chair that is next to the TV, we need to identify the chair and the TV in the scene.

The TV is located at obj_9 with the box coordinates [3.9, 4.1, 1.5, 0.6, 0.4, 0.5].
The chair is located at obj_2 with the box coordinates [4.4, 3.7, 0.6, 0.7, 0.7, 0.9].
However, this chair is not next to the TV.

The next chair is located at obj_3 with the box coordinates [5.3, 3.2, 0.5, 0.6, 0.6, 0.9].
This chair is indeed between the table and the TV.

The table is located at obj_8 with the box coordinates [4.9, 3.4, 0.5, 1.0, 1.0, 0.9].

Answer: **obj_3.**

Base Model

The objects that might be related to the provided caption are:

- obj_2: Chair at (4.4, 3.7, 0.6) with dimensions (0.7, 0.7, 0.9).
- obj_3: Chair at (5.3, 3.2, 0.5) with dimensions (0.6, 0.6, 0.9).
- obj_9: TV at (3.9, 4.1, 1.5) with dimensions (0.6, 0.4, 0.5).
- obj_8: Table at (4.9, 3.4, 0.5) with dimensions (1.0, 1.0, 0.9).

...

The caption describes a chair next to the TV, between the table and the TV. Given the 2D positions:

- obj_2 is very close to obj_9 (TV) and lies between obj_8 (table) and obj_9 (TV).
- obj_3 is closer to obj_8 (table) and not directly next to obj_9 (TV).

Thus, obj_2 is the chair next to the TV, between the table and the TV.

Answer: **obj_2.**

Reason3DVG



The chair that is most in the middle of the room.
(Out-of-domain, unseen query)

To find the object ID of the chair in the middle of the room, we need to identify which chair is in the middle.

...

To confirm that this is the middle chair, we need to look at the x and y coordinates of the chairs. obj_10 has an x coordinate of 1.4, which is relatively close to the middle of the room.

...

Based on this information, we can conclude the object ID of the dark color chair in the middle of the room is:

Answer: **obj_10.**

Base Model

The relevant objects are those with the name "chair" which are:

- obj_9: "chair", box: 3.7, 1.2, 0.4, 0.7, 0.7, 0.8
- ...
- obj_12: "chair", box: 3.0, 1.9, 0.6, 0.7, 0.6, 0.5

...

Assuming the current location is the center of "floor" which is at (2.8, 2.3).

To determine which chair is in the middle of the room, we can calculate the Euclidean distance of each chair ...

1. Distance from obj_9 (3.7, 1.2):

$$\text{Distance} = \sqrt{(3.7 - 2.8)^2 + (1.2 - 2.3)^2} = \sqrt{0.9^2 + 1.1^2} = \sqrt{0.81 + 1.21} = \sqrt{2.02} \approx 1.42$$

...

4. Distance from obj_12 (3.0, 1.9):

$$\text{Distance} = \sqrt{(3.0 - 2.8)^2 + (1.9 - 2.3)^2} = \sqrt{0.2^2 + 0.4^2} = \sqrt{0.04 + 0.16} = \sqrt{0.2} \approx 0.44$$

The closest chair to the assumed location is obj_12 with a distance of approximately 0.44.

Answer: **obj_12.**

Reason3DVG

Figure 4. Qualitative results from NR3D. Green boxes and red boxes indicate predictions from our Reason3DVG and the base model, respectively. All predicted bounding boxes are re-plotted for better clarity.

captioner with richer semantic information can enhance the LLM’s reasoning process.

6. Conclusion

In this work, we propose an automatic data pipeline for generating 3D visual grounding data with reasoning supervision, requiring no human annotations. Using our col-

lected data, we perform LLM fine-tuning and introduce Reason3DVG, a strong 3D visual grounding LLM. Reason3DVG outperforms multiple SOTA proprietary LLM-based methods on ScanRefer and NR3D, and surpasses 3D-GRAND using only 1.6% of its training data. Our results show that detailed reasoning supervision, not just data scale, is key to advancing 3D visual grounding with LLMs.